IB Computer Science

26 March 2026

Criterion E: Evaluation

Meeting the Criteria for Success:

The application successfully loads and renders 3D models in .dae, .obj, .fbx, .gltf, and .glb formats, correctly - This is met

The app correctly processes models into meshes, vertices, indices, and texture coordinates - This is met

The application supports texture mapping and models display with accurate materials - This is met

The program demonstrates a clear model loading pipeline including file importing, node traversal, mesh processing, and rendering - This is met

The application loads textures efficiently using a cache that prevents the same texture from being uploaded to the GPU more than once - This is met

The program renders models at an interactive frame rate, shown by the FPS counter in the corner of the window - This is met

The code is modular, with the entire loading and rendering pipeline contained in a single includable header, making it easy to build on later - This is met.

The application provides visible evidence of correct functionality through testing with multiple different model formats and test models - This is met.

Improvements in Future:

Overall, I think the project is a success. Loading models and seeing them render correctly in real time is a pretty satisfying result, especially the wireframe view. Knowing all the small details of the project and important steps and processes I went through to get a working model loader was quite a journey. Although there are still some things I want to add.

The biggest one is that texture loading is not fully reliable across all model formats. Most formats work well, but embedded textures in some FBX files don't always load correctly. This doesn't affect the main functionality for most models, but it is something that would need to be addressed before this could be used more broadly.

Another limitation is that the program only supports basic lighting. There is no support for point lights or spotlights, and there is no shadow rendering. For the purposes of this project that was acceptable, but it does mean that visually it's less realistic than something like a full

PBR pipeline. Adding shadow maps would be the most meaningful visual improvement for a future version.

The program also has no way to inspect a model beyond looking at it. Something like a mesh info panel showing vertex count, triangle count, and material names would make it more useful as an actual tool rather than just a viewer. This was something I considered adding but ran out of time for.

Finally, it's true that this model loader is built with modular classes, there currently isn't a way to load more than one model at a time, or make any kind of scene. Supporting a small scene with multiple models or objects would be a good place to go next, as it probably wouldn't require a bunch of large changes to the existing structure. All the things I've learned along the way, like linear algebra, GPU memory management, and general rendering and computer graphics. This project could become something genuinely useful, and already is quite useful for anyone wanting to look at 3D models without opening some large program like Blender.